LAPORAN PRAKTIKUM PEMROGRAMAN BERBASIS OBJEK

Minggu Ke-2

disusun Oleh:

Az Zahrand Solichul Tajussalathin

NIM 2511532001

Dosen Pengampu: Nurfiah S.ST, M.Kom

Asisten Pratikum: Siti Fadhilah Rahmi



FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2026

DAFTAR ISI

DAFTAR ISI ..... **i**

KATA PENGANTAR ..... **ii**

BAB I PENDAHULUAN **1**

1.1 Latar Belakang **1**

1.2 Tujuan Praktikum **1**

1.3 Manfaat Praktikum **1**

BAB II PEMBAHASAN **2**

2.1 Implementasi Class Diagram Transaksi **2**

2.2 Implementasi Relasi pada Rekening.java **2**

2.3 Implementasi Pencatatan Riwayat Transaksi **3**

BAB III TUGAS **5**

3.1 Integrasi Rekam Jejak Penarikan **5**

3.2 Implementasi Fitur Cetak Mutasi Rekening **5**

BAB IV KESIMPULAN **7**

4.1 Ringkasan Hasil Praktikum **7**

DAFTAR PUSTAKA ..... **iii**

KATA PENGANTAR

Puji syukur kehadirat Allah SWT atas limpahan rahmat dan karunia-Nya sehingga laporan praktikum ini dapat disusun dan diselesaikan dengan baik. Laporan ini disusun sebagai salah satu bentuk pelaksanaan kegiatan praktikum pada mata kuliah Pemrograman Berbasis Objek.

Ucapan terima kasih disampaikan kepada Bapak Nurfiah S.ST, M.Kom. selaku dosen pengampu, serta kepada asisten laboratorium yang telah memberikan bimbingan selama kegiatan praktikum berlangsung.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna, baik dari segi penulisan maupun isi. Oleh karena itu, saran dan kritik yang bersifat membangun sangat diharapkan demi perbaikan di masa yang akan datang. Harapannya, laporan ini dapat memberikan manfaat dan menjadi referensi bagi pembelajaran dalam bahasa pemrograman Java.

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Dalam Pemrograman Berorientasi Objek (PBO), perangkat lunak dimodelkan berdasarkan entitas yang ada di dunia nyata. Dua pilar paling mendasar dalam pemodelan ini adalah Class dan Object. Class berfungsi sebagai cetak biru (blueprint) dari sebuah objek yang mendefinisikan struktur data (atribut atau state) serta sekumpulan operasi (metode atau behavior) yang dapat dilakukan pada data tersebut. Class itu sendiri tidak memakan ruang memori untuk menyimpan nilai data, melainkan hanya berfungsi sebagai tipe data referensi. Di sisi lain, Object adalah instansiasi konkret dari sebuah class yang dialokasikan di dalam memori komputer, di mana setiap objek memiliki state atau kondisi yang saling independen. Selain itu, PBO juga menggunakan Constructor, yaitu blok kode khusus yang dieksekusi secara otomatis pada saat objek dibuat untuk menginisialisasi state awal dari objek tersebut.

1.2 Tujuan Praktikum

Tujuan dari praktikum ini adalah:

- Mendefinisikan class sebagai model dari entitas dunia nyata.
- Mendefinisikan object dan memahami alokasi memori independen.
- Menggunakan constructor untuk inisialisasi awal state pada objek.

1.3 Manfaat Praktikum

Manfaat yang diperoleh dari praktikum ini antara lain:

- Meningkatkan pemahaman mengenai manajemen memori dinamis di Java.
- Meningkatkan fondasi logika untuk membangun struktur data dan arsitektur program yang lebih kompleks di masa depan.

BAB 2

PEMBAHASAN

2.1 Implementasi Class Diagram Transaksi

```
package praktikumMinggu1;

public class Transaksi {
	String idTransaksi;
	String jenis;
	double nominal;

	// constructor
	public Transaksi(String id, String jenis, double nominal)
	{
		this.idTransaksi = id;
		this.jenis = jenis;
		this.nominal = nominal;
	}
	public void cetakDetail() {
		System.out.println("ID: " + idTransaksi + " | Jenis: " + jenis
+ " | Nominal: Rp" + nominal);

	}
}
```

Relasional dimulai dengan membuat class Transaksi yang berperan sebagai cetak biru untuk entitas riwayat. Class ini didefinisikan dengan tiga buah atribut dasar, yaitu idTransaksi (String), jenis (String), dan nominal (double). Konstruktor pada kelas ini bertugas menerima parameter input dan menginisialisasi nilai awal untuk ketiga atribut tersebut pada saat objek dibuat. Untuk mendukung fitur pelaporan, sebuah method bernama cetakDetail() diimplementasikan untuk mencetak satu baris informasi detail transaksi (ID, Jenis, dan Nominal)

2.2 Implementasi Relasi pada Rekening.java

```
package praktikumMinggu1;

import java.util.ArrayList;

public class Rekening {
    String nomorRekening;
    String namaPemilik;
    double saldo;

    // Implementasi asosiasi
```

```
    ArrayList<Transaksi> riwayatTransaksi;


    public Rekening(String nomor, String nama, double saldoAwal) {
        this.nomorRekening = nomor;
        this.namaPemilik = nama;
        this.saldo = saldoAwal;

        //inisialisasi arraylist didalam constructor
        this.riwayatTransaksi = new ArrayList<>();

        System.out.println("Rekening atas nama " + namaPemilik + " berhasil
dibuat dengan saldo Rp" + saldo);
    }
```

Relasi asosiasi one-to-many dibentuk dengan menyematkan atribut riwayatTransaksi bertipe ArrayList<Transaksi> ke dalam class Rekening. Agar tidak terjadi NullPointerException ketika objek rekening baru dialokasikan ke memori, koleksi tersebut diinisialisas sebagai new ArrayList<>() di dalam konstruktor utama. Sebagai contoh pada operasi penyetoran, program memanipulasi saldo sekaligus menginstansi objek Transaksi baru (bertipe "Kredit") yang kemudian dimasukkan langsung ke dalam koleksi menggunakan fungsi .add()

2.3 Implementasi Pencatatan Riwayat Transaksi

```
public void setorTunai(double nominal) {
        if (nominal > 10000) {
            saldo += nominal;
            // merekam riwayat
            String idTrx = "TRX-S-" + System.currentTimeMillis();
            Transaksi trxBaru = new Transaksi(idTrx, "Kredit", nominal);
            riwayatTransaksi.add(trxBaru);

            System.out.println("Setor tunai Rp" + nominal + " berhasil.
Saldo saat ini: Rp" + saldo);
        } else if (nominal < 10000) {
            System.out.println("Gagal: Nominal setor harus lebih dari Rp.
10.000!");
        } else {
            System.out.println("Gagal: Nominal setor harus lebih dari Rp.
10.000!");
        }

    }
```

Proses pencatatan riwayat mutasi diimplementasikan secara langsung di dalam metode operasional setorTunai dan tarikTunai pada class Rekening. Mekanisme

pencatatan ini bertumpu pada instansiasi objek Transaksi baru, yang logikanya diletakkan setelah blok validasi saldo dan hanya dieksekusi apabila transaksi dinyatakan berhasil. Dalam proses pembuatan objek riwayat tersebut, program mengonstruksi ID transaksi dengan fungsi bawaan Java, yaitu System.currentTimeMillis(). Nilai waktu tersebut kemudian digabungkan (concatenation) dengan awalan (prefix) string yang berbeda untuk mengkategorikan transaksi

BAB 3

TUGAS

3.1 Integrasi Rekam Jejak Penarikan

```
    public void tarikTunai(double nominal) {
      if (nominal < 10000) {
            System.out.println("Penarikan gagal! Minimal penarikan saldo
adalah Rp10.000!");
      } else if (saldo < nominal) {
            System.out.println("Penarikan gagal! Saldo anda tidak
mencukupi, saldo anda saat ini adalah Rp:" + saldo);
      } else {
            saldo = saldo - nominal;

           // Tugas 1
           String idTrx = "TRX-T-" + System.currentTimeMillis();
           Transaksi trxBaru = new Transaksi(idTrx, "Debit", nominal);
           riwayatTransaksi.add(trxBaru);

            System.out.println("Tarik tunai Rp" + nominal + " berhasil.
Saldo saat ini: Rp" + saldo);
      }
    }
```

Logika perekaman ini disisipkan di dalam kondisi validasi if-else, secara spesifik hanya dieksekusi apabila penarikan dinilai berhasil (saldo mencukupi). Objek Setelah objek berhasil dibangun, sistem akan menambahkannya ke dalam history riwayatTransaksi milik akun yang bersangkutan

3.2 Implementasi Fitur Cetak Mutasi Rekening

```
    // Tugas 2
    public void cetakMutasi() {
        System.out.println("--- MUTASI REKENING ---");
        if (riwayatTransaksi.isEmpty()) {
            System.out.println("Belum ada transaksi pada rekening ini");
        } else {
            for (Transaksi trx : riwayatTransaksi) {
                trx.cetakDetail();
            }
        }
        System.out.println("----------------------");

    }
```

Pembuatan fitur cetak mutasi diselesaikan dengan menyusun method cetakMutasi() di dalam kelas Rekening. Struktur kontrol perulangan for-each

dimanfaatkan untuk menelusuri (traverse) setiap entitas Transaksi di dalam ArrayList. Pada setiap iterasinya, program akan mengeksekusi metode cetakDetail() dari objek yang bersangkutan untuk menampilkan rincian data ke layar. Apabila penelusuran tidak menemukan satu pun data (koleksi kosong), sistem mengeluarkan teks bertuliskan "Belum ada transaksi pada rekening ini"

BAB 4

KESIMPULAN

4.1 Ringkasan Hasil Praktikum

Dari pelaksanaan praktikum modul kedua, dapat ditarik simpulan bahwa implementasi relasi antar objek (asosiasi) penting dalam menyusun sistem yang terstruktur. Penggunaan tipe data ArrayList membuktikan bahwa sebuah objek utama (Rekening) dapat menyimpan, mengelola, dan memanipulasi sekumpulan objek independen (Transaksi). Integrasi antara pencatatan riwayat dengan proses validasi saldo pada method penarikan dan penyetoran juga menjamin validitas rekam jejak sistem. Terakhir, fleksibilitas navigasi memori ditunjukkan melalui pemanfaatan metode for-each loop maupun manipulasi indeks for-loop dalam mengekstrak dan menampilkan detail mutasi berdasarkan kebutuhan fungsional program

DAFTAR PUSTAKA

[1] Nurfiah. Modul Praktikum PBO - Java: Praktikum 2 Relasi Antar Objek (Class Diagram & Asosiasi). [Diakses: 17 September 2026].